RAN Sentinel: technical documentation

Oct 28, 2026 · @QuanTell

1. Overview

RAN Sentinel is an AI co-engineer for the operations centre of a mobile network. It watches a cluster of LTE cells and spots one that is degrading. Meta's Muse Spark agents investigate the cell with read-only tools and propose a fix. Fixed safety rules and a digital twin check that fix before any engineer sees it. The recommendation then goes to the one team that owns it. When a configuration change is approved, it goes out as an Open M-Plane change package with automatic rollback, and the cell is watched until it recovers. Every step is written to the **Evidence-to-Action Safety Ledger**.

It was built by AKMaasif (QuanTell), an LTE RAN engineer, for the Meta Global AI Developer Hackathon (Indie track, theme "Build for Your World"). The code is MIT-licensed.

The problem

US regional and rural carriers run the same kind of LTE network as the national carriers, with a much smaller operations team. For each degraded cell an engineer opens KPI dashboards, alarms and the radio configuration, works out the cause, and decides who should act. A wrong power or tilt change can harm neighbouring cells. Outages also carry federal reporting duties under FCC Part 4 (47 CFR 4.9).

The US setting

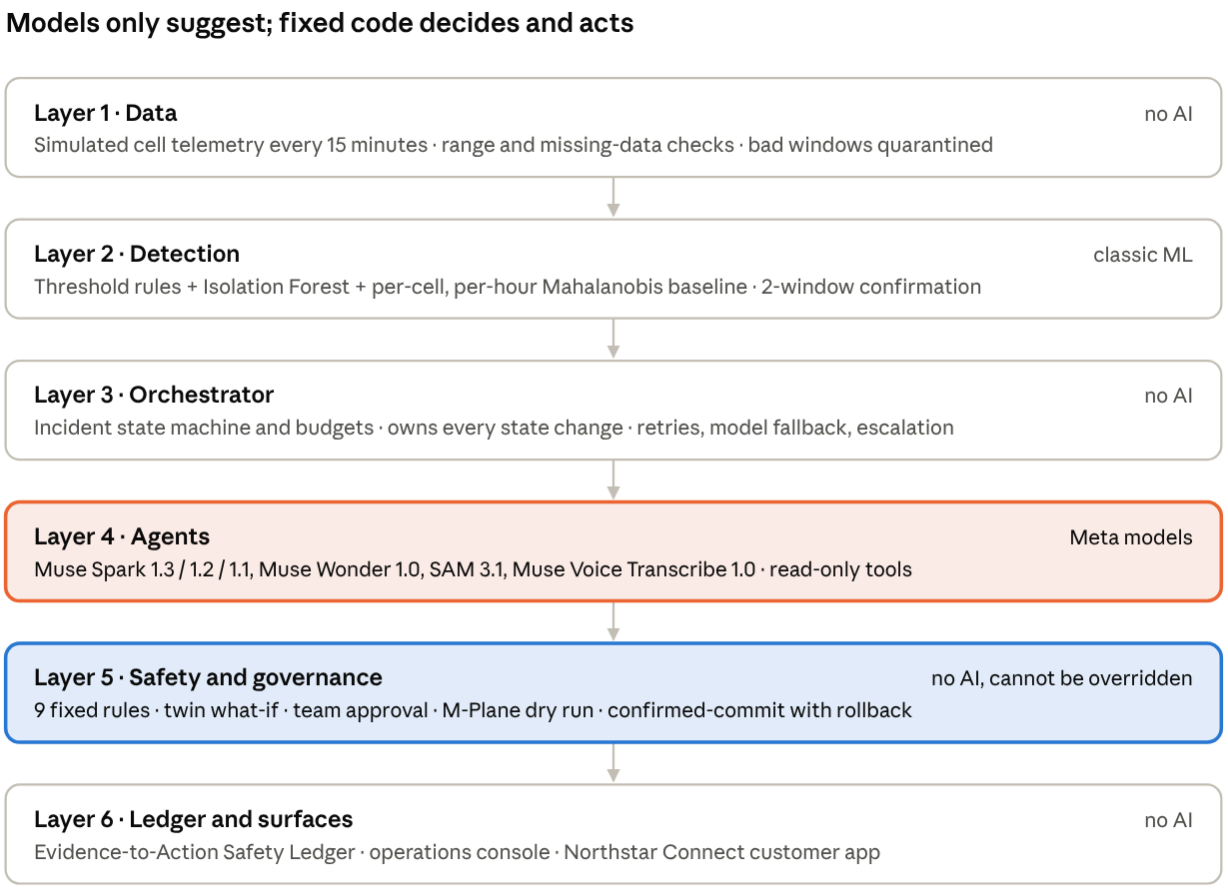
- **Network:** 36 simulated LTE cells (12 sites, 3 sectors each) in Seattle, WA, on US bands B66, B2 and B71.
- **Places:** real Seattle neighbourhoods and ZIP codes, such as Downtown 98101 and Queen Anne 98119.
- **Time:** Pacific time throughout.
- **Regulation:** an FCC Part 4 outage check (section 8).
- **Operator:** the fictional carrier **Northstar Connect**, whose customers see a plain-language status app.

What it is not

Everything runs on a simulated network; nothing connects to a real operator. The FCC check is decision support, not legal advice. No endorsement by Meta, any operator or any equipment vendor is implied.

2. Architecture

RAN Sentinel has six layers. Only layer 4 uses Meta's models, and only layer 3 changes an incident's state. A model can suggest a cause or a plan; it can never apply a change, approve one or skip a rule.



architecture · six layers, AI only in layer 4

Components

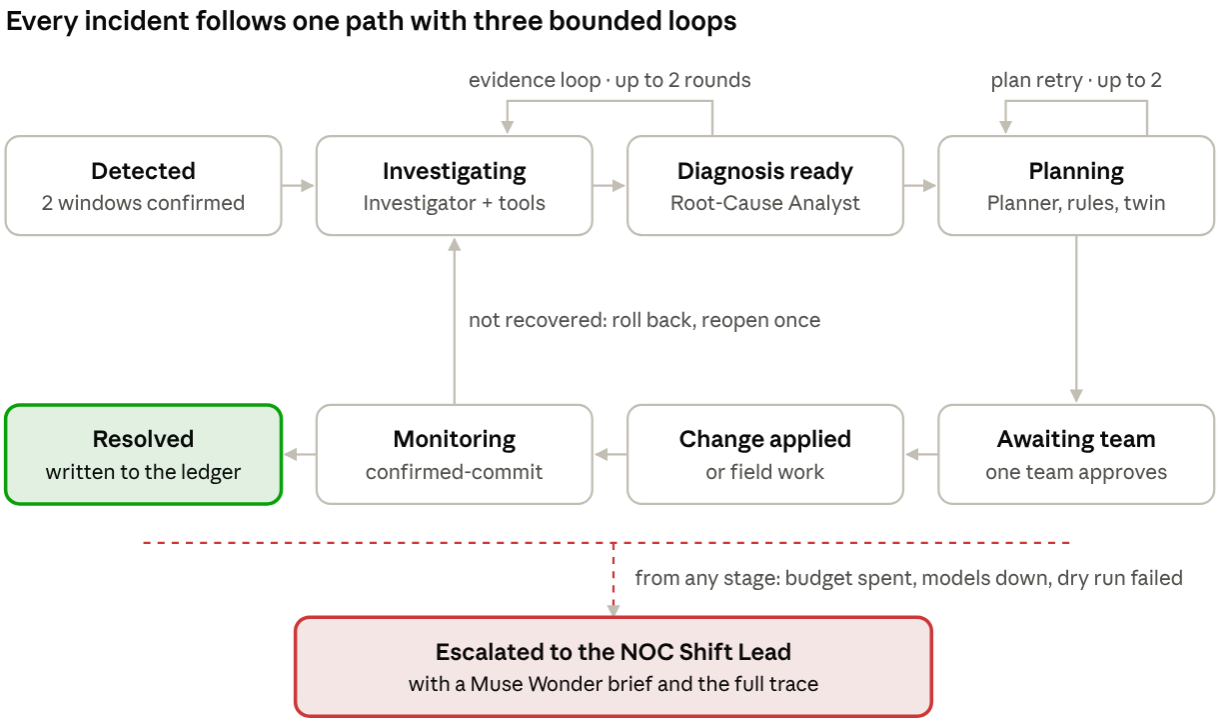
Component	File	Job
Network simulator	`backend/app/simulator.py`	36 Seattle cells, KPIs every 15 minutes, five injectable fault types, alarms

Component	File	Job
Detection	`backend/app/detection.py`	Data checks, three detectors, two-window confirmation
Orchestrator	`backend/app/orchestrator.py`	State machine, budgets, loops, routing, tickets, photos, handover
Agents	`backend/app/agents.py`	Prompts, JSON schemas, citation checks and the offline rules engine
Model provider	`backend/app/providers.py`	Meta Model API client: chat, tools, vision, SAM 3.1, voice; retry and fallback chain
Read-only tools	`backend/app/tools.py`	Anomaly result, KPI window, alarm history, cell configuration, diagnostic checks
Safety validator	`backend/app/safety.py`	Allow-list, nine rules, department routing
Twin	`backend/app/twin.py`	What-if simulation of a change; Maveric adapter
Open M-Plane	`backend/app/mplane.py`	NETCONF change packages, dry run, confirmed-commit and rollback on a simulated radio unit
FCC check	`backend/app/regulatory.py`	Part 4 outage user-minutes and deadlines
Store and ledger	`backend/app/store.py`	SQLite: tickets, ledger, customer updates
API	`backend/app/main.py`	FastAPI endpoints for both apps
Ops console	`frontend/ops/`	Engineer console (vanilla JavaScript)
Customer app	`frontend/connect/`	Northstar Connect status page and assistant

The stack is Python 3.10+ with FastAPI, Uvicorn, NumPy, scikit-learn, httpx and SQLite. The OpenAI Python SDK is the client for Meta's OpenAI-compatible endpoint at `https://api.meta.ai/v1`. The front end is plain HTML, CSS and JavaScript with the IBM Plex fonts, with no build step.

3. End-to-end workflow

The orchestrator owns the incident state machine. The model can suggest; only this code decides a transition.



incident lifecycle · 8 states, 3 loops, 1 side exit

Step by step

1. **Data.** Every 15 minutes the simulator emits KPIs for all 36 cells. Values out of range or missing are quarantined and logged as a data-quality issue; they never reach the detector.
2. **Detection.** Three detectors score each cell (section 6). An alert needs two consecutive windows. A cell with an open incident attaches the new alert to it; neighbouring cells hit at the same time are grouped as related.
3. **Investigation.** The **Investigator** (Muse Spark 1.3) calls five read-only tools: `get_anomaly_result`, `get_kpi_window`, `get_alarm_history`, `get_cell_configuration` (an M-Plane get-config) and `run_diagnostic_check`. A failed tool call is marked as missing evidence, never invented.
4. **Diagnosis.** The **Root-Cause Analyst** ranks four causes (capacity congestion, overshooting coverage, external interference, hardware fault). Every claim must cite an evidence id. If evidence is thin, it asks for more read-only checks from a fixed list: neighbour load, timing-advance distribution, uplink interference scan, VSWR and power readback, config against plan, and handover statistics.

5. **Planning.** The **Remediation Planner** proposes one action (power change, tilt change, or no config change) with prerequisite checks and a rollback. The nine safety rules and the twin check it (section 4), and the **Plan Reviewer** on Muse Spark 1.2 gives an independent second opinion. A failed plan goes back to the planner with the reasons.
6. **Routing.** The plan goes to exactly one team: RAN Optimisation (power or tilt), RF Engineering (interference), Field Operations (hardware), NOC Data Platform (data), NOC Shift Lead (escalations) or Regulatory Compliance (FCC). Nothing changes until that team approves, rejects or asks for more information.
7. **Action.** An approved config change is sent as an Open M-Plane NETCONF package with a dry run first and a confirmed-commit. Field work opens a dispatch ticket instead; the technician can add a site photo.
8. **Monitoring.** The cell is watched. If it recovers, the commit is confirmed and the incident resolves. If not, the radio rolls back by itself and the investigation reopens once.
9. **Records and comms.** Every step goes to the Safety Ledger. The **Customer Comms** agent writes plain-language updates, which a filter checks for internal details before publishing. The **Incident Reporter** writes the report, and **Muse Wonder** writes the shift handover.

Data-quality incidents take a short path: the system re-polls the collector by itself (`AUTO_REMEDIATION`), and only a lasting dropout goes to the NOC Data Platform team.

Budgets and the four loops

Loop	What repeats	Limit
Evidence	Analyst asks for more read-only checks	2 rounds, 8 tool calls per incident
Output repair	Agent JSON fails its schema or cites unknown evidence	1 repair try
Plan revision	Validator or reviewer reasons go back to the planner	2 retries
Recovery	Cell does not recover after a change; rollback and reopen	1 reopen

Error paths

What goes wrong	What happens
Bad or missing telemetry	Window quarantined; a lasting dropout becomes a data-quality incident
Duplicate or related alert	Attached to the open incident, or marked related
Meta API error or timeout	Retry with backoff, then the next model in the chain (1.3, 1.2, 1.1); if all fail, escalate
Invalid model output	One repair try, then escalate
Budget spent	Escalate with the full trace and a Muse Wonder brief
Unsafe plan	Rejected with reasons; replan up to twice, then escalate
Twin unavailable	Plan flagged "not simulated" and needs engineer review
M-Plane dry run fails	Escalate; nothing is committed
Commit not confirmed	The radio reverts by itself
Customer text leaks internal details	Blocked by the filter and replaced by a safe template
Prompt injection inside data	Tool data is passed as data, and the schema and citation checks reject output that follows it

4. Safety and the Safety Ledger

Safety is layer 5. It is plain code with no model, and nothing an agent writes can switch it off. A plan reaches an engineer only after all of these checks.

The nine rules (`backend/app/safety.py`)

#	Rule	What it checks
1	Action in allow-list	Only `adjust_tx_power` , `adjust_tilt` or `no_config_change`
2	Targets the incident cell	A config change must be on the cell under investigation

#	Rule	What it checks
3	Fits the diagnosis	Power or tilt only for capacity or overshoot; interference and hardware get no config change
4	Step within limit	Power up to 3 dB per step, tilt up to 2 degrees
5	Result within bounds	Power 37 to 46 dBm, tilt 0 to 12 degrees
6	Rollback restores current value	The rollback value equals the running value, read over M-Plane, and has a trigger
7	Read-only checks first	At least one prerequisite check from the read-only list
8	Evidence cited	Every cited evidence id exists in this incident
9	Sandbox impact	The twin predicts coverage loss of 1% or less and neighbour PRB use of 85% or less

The other gates

- **Digital twin what-if.** The change is simulated on a copy of the network first. If the twin is down, rule 9 shows "not run" and the plan is flagged for engineer review.
- **Plan Reviewer.** Muse Spark 1.2 reads the plan as an independent second opinion. Its concerns go to the engineer; it cannot approve.
- **Team approval.** Exactly one team receives the plan as a ticket and can approve, reject or ask for more information.
- **M-Plane dry run.** The NETCONF package is validated on the candidate datastore before commit.
- **Confirmed-commit.** The commit carries a confirm timeout of 90 simulated minutes. Monitoring confirms it only if the cell recovers; otherwise the radio rolls back by itself.
- **Red-team switch.** In the demo, a switch forces an aggressive first plan so judges can watch the rules block it and the planner revise it.

The Evidence-to-Action Safety Ledger

Every step is appended to a SQLite ledger: sequence number, incident, wall-clock and simulated time, actor, event, summary and the full payload. Actors include the detector, investigator, analyst, validator, twin, reviewer, orchestrator, monitor, reporter, briefer, compliance and the engineer. Each tool call, model call, rule result, decision, M-Plane message and customer update can be traced from evidence to action. The ledger page in the console filters it by incident and shows the Muse Wonder shift handover.

5. Meta models

Each model has one job where it helps. All of them are called through the Meta Model API at `https://api.meta.ai/v1` with the key from `MODEL_API_KEY` in `.env`.

Model	Agents and jobs	Why this model	If it is unavailable
muse-spark-1.3	Investigator (tool calls), Root-Cause Analyst, Remediation Planner, Customer Comms, Customer Assistant, Incident Reporter, photo reading	Tool calls and JSON output passed on the key	Falls back to 1.2, then 1.1
muse-spark-1.2	Plan Reviewer; fallback for every agent	A different model gives a more independent second opinion	Falls back to 1.3, then 1.1
muse-spark-1.1	Last-resort tier	One more chance before an incident escalates to a person	Escalate to the NOC Shift Lead
muse-wonder-1.0	Escalation Briefer; Shift Handover	The hardest cases get the strongest model; the handover reads every incident and the whole ledger, which needs the 1M-token context	Muse Spark 1.3, then a template
sam-3.1	Field Photo Analyst: boxes antennas, cables and connectors in a technician's site photo	Segmentation is SAM's job; Spark then reads the photo with those boxes	Photo is stored without analysis
muse-voice-transcribe-1.0	Dictated decision notes (ops) and spoken questions (customers)	Engineers on a mast or in a van have their hands full; cell IDs and site names go as keyword hints	Type instead

How the calls work

- **Chat and tools** use the OpenAI-compatible chat completions endpoint. Agent output must be JSON that passes its schema and cites only real evidence ids, with one repair try.

- **Fallback chain.** Each call retries with backoff, then moves down the chain. A three-tier chain (1.3, 1.2, 1.1) runs before escalation.
- **SAM 3.1** uses `POST /v1/responses` with a text prompt and the image as a data URL. It returns boxes as tokens, which the app parses into coordinates with confidence. It tries the request with confidence metadata first and remembers which request shape worked.
- **Voice** uses `POST /v1/asr/transcribe`, a multipart request with a JSON part (model, push-to-talk mode, WAV, keywords) and a mono 16-bit WAV recorded in the browser.

Advisory by design

Voice and photos never change an incident. A dictated note can highlight a decision button (approve, reject, more info, resolved, filed), but the engineer still clicks it. A photo adds evidence and can raise a "does not match the diagnosis" warning.

Deliberately not used

- **muse-image-1.0.** A generated picture is not evidence and could be mistaken for a real site photo.
- **The -contributor variants.** They cost less, but Meta may train on what is sent, and incidents and customer questions can hold sensitive data. An operator can opt in for the customer assistant only with `ASSISTANT_MODEL=muse-spark-1.3-contributor`.

Offline mode

Without a key, the agents use an offline rules engine so the app still runs. The console then says "Offline rules engine" everywhere, so those results cannot be mistaken for Meta-powered ones.

`scripts/check_meta.py` tests every model through the same code the app uses: chat, tool calls, JSON, vision, SAM with a generated image, and transcription with a generated WAV. It prints only the last four characters of the key. Add `--image site.jpg --audio note.wav` to try your own photo and recording.

6. Detection and benchmarks

Detection uses no language model. It reads eight KPIs per cell: PRB utilisation, downlink throughput, CQI, SINR, uplink RSSI, drop rate, handover success and the share of users far from the site. Any of three detectors can raise an alert, and an alert fires only after two consecutive 15-minute windows.

1. **Threshold rules:** seven limits an engineer would set. PRB use above 90%, drop rate above 2%, handover success below 97%, SINR below 6 dB, uplink RSSI above -96 dBm, far-user share above 18%, availability below 99.9%.
2. **Isolation Forest:** learns normal traffic across the cluster.
3. **Cell baseline:** compares each KPI with what is normal for that cell at that hour, scored with a Ledoit-Wolf Mahalanobis distance.

None of them needs labelled faults.

Open dataset: TelecomTS

TelecomTS (MIT licence, ICML 2026) has 32,000 labelled 5G windows with 11 anomaly types. The split was 80/20 train and test. The unsupervised detector was chosen on a validation split taken from the training data, never on test, and thresholds were set for a 1% false-alarm rate. The benchmark ran on your PC because Hugging Face is blocked from the cloud environment.

Detector (TelecomTS test set)	Labels needed	F1	Recall	ROC-AUC	False alarms
Isolation Forest	No	0.23	0.16	0.76	0.9%
PCA reconstruction, context baseline	No	0.52	0.47	0.91	1.3%
Mahalanobis, context baseline (selected)	No	**0.61**	**0.59**	**0.92**	1.4%
Gradient-boosted trees (reference only)	Yes	0.97	0.94	1.00	0.0%

With labels, root-cause classification scored 99.0% accuracy across the 10 synthetic types, and 98.9% after mapping to RAN Sentinel's four causes. These supervised scores are likely optimistic, because TelecomTS anomalies were recorded in separate sessions from normal traffic. Jamming is the hardest type to catch without labels: only 15% was detected. Because labels help so much, the roadmap turns incidents that engineers confirm in the ledger into training labels.

Simulator benchmark

48 injected faults across the five fault types, plus 108 normal cell-days.

Detector	Faults detected	False alarms in 108 normal cell-days
Threshold rules only	79.2%	0
Isolation Forest only	56.2%	0
Cell baseline only	64.6%	0
All three (shipped)	**100% (48 of 48)**	**0**

After the full investigation, the correct root cause ranked first in 97.9% of simulator runs.

Reproduce

```
pip install huggingface_hub
py scripts\benchmark_telecomts.py --export-features
data\telecomts_features.csv.gz
py scripts\benchmark_telecomts.py --features data\telecomts_features.csv.gz
py scripts\benchmark_simulator.py
```

The first command downloads about 1.1 GB once. Later runs use the compact feature file.

7. Open M-Plane and Maveric

Both projects are Meta contributions to LF Connectivity. RAN Sentinel follows their designs, but neither is bundled, and the demo makes clear what is simulated.

	Open M-Plane (`lf-connectivity/open-mplane`)	Maveric (`lf-connectivity/maveric`)
What it is	O-RAN M-Plane NETCONF/YANG stack, in C, MIT licence	AI/ML RAN optimisation testbed with an RF digital twin, in Python, MIT licence
Role in RAN Sentinel	The vendor-neutral way to read radio config and package an approved change	The what-if sandbox for every plan (safety rule 9)

	Open M-Plane (`lf-connectivity/open-mplane`)	Maveric (`lf-connectivity/maveric`)
What is real	The message sequence: `<get-config>` , `<edit-config>` to the candidate datastore, `<validate>` , then `<commit>` with `<confirmed/>` and a confirm timeout. If nobody confirms, the radio reverts. The XML package is shown in the incident.	An adapter that checks whether a Maveric service at `MAVERIC_URL` is reachable
What is simulated	The radio unit is in-process. The XML uses simplified O-RAN-style element names, not the full o-ran-* YANG paths. Electrical tilt is changed directly; real sites often use RET/AISG for tilt.	The prediction itself runs on a copy of the simulator ("fallback twin"), and the result says so. Wiring Maveric's train and simulate calls needs a trained twin of the demo area.

The twin predicts the cell's coverage loss and the offload neighbour's PRB use before and after the change. The limits are coverage loss of 1% or less and neighbour PRB use of 85% or less.

8. FCC Part 4 outage check

`backend/app/regulatory.py` gives a US network operations centre decision support for 47 CFR 4.9(e), wireless outage reporting through NORS.

- **What counts.** Only incidents where the cell loses service (a hardware fault, or availability below 99.9% on a network incident). Slower service is not an outage under Part 4.
- **User-minutes.** Users potentially affected times minutes since discovery. One sector counts as a third of a macro site's average users, set with `FCC_USERS_PER_SITE` in `.env` (default 1,500).
- **Threshold.** At least 30 minutes and at least 900,000 user-minutes makes it reportable.
- **Action.** A reportable outage opens a Regulatory Compliance ticket with three deadlines from discovery: notification within 120 minutes, initial report within 72 hours, final report within 30 days. The incident's FCC panel shows the running count and the deadlines, and a dictated note such as "NORS notification submitted" can mark it filed.

This is decision support for the NOC, not legal advice. With the default of 1,500 users per site, a single sector does not reach the threshold in a short demo; raise `FCC_USERS_PER_SITE` to show a reportable case.

9. Screens

Operations console (`/ops/`)

The console follows the layout of professional network operations (OSS) tools such as Nokia NetAct or Ericsson ENM, without copying any vendor's branding:

- a **top bar** with global search (incident ID, cell ID, site name or ZIP code) and live alarm counters by severity (critical, major, minor, warning);
- a dark **module rail** grouped as Monitor, Operate and Assure;
- one scrolling **work area**;
- a **Demo controls** drawer that opens beside any page;
- a **status bar** with the telemetry window, AI calls and average latency, the twin and the M-Plane link.

Module	What it shows
Monitor · Network overview	KPI tiles with trend sparklines, the cluster map with sectors coloured by health, the incident work queue, network health chart, AI activity board and a live model-call feed, top cells by anomaly score
Monitor · Alarm monitor	Every alarm with X.733-style fields: severity, event type, probable cause, managed object, site, band, ZIP, linked incident. Filter by severity or text, and acknowledge (the ack records who and when)
Monitor · Cell explorer	A network tree by site and band. For each cell, tabs for KPI charts against their limits, alarms, M-Plane radio config against plan, and detector scores
Operate · Incidents	The incident list, and the incident workspace: a fact bar, the workflow timeline, and tabs for diagnosis, plans and safety checks, evidence, field photos, an agent console (a terminal window of every model and tool call), report and ledger. A side column holds the decision panel, detection, FCC and radio config
Operate · Approvals	One card per team with its queue, and the tickets waiting on each
Assure · Safety ledger	The full ledger with an incident filter, and the Muse Wonder shift handover

Module	What it shows
Assure · Model card	Each model's role and fallback, the models not used and why, and the measured accuracy

The backend adds `GET /api/alarms` (the last 400 alarms with event type and probable cause) and `POST /api/alarms/{id}/ack` for the alarm monitor. The layout was checked at desktop (1440 px), tablet (1024 px) and phone (390 px) widths with no horizontal scrolling.

Northstar Connect customer app (`/connect/`)

- **Service status** for Seattle in Pacific time: a ZIP code check, a banner when there is a problem in the customer's area, active updates and the status of every neighbourhood.
- **Ask us:** the Customer Assistant answers questions such as "Is there an issue in 98101?" from published updates only. Customers can type or speak the question.
- Every customer text passes a filter that blocks cell IDs, dBm, tilt, PRB, VSWR, M-Plane, NETCONF, alarm codes and hypotheses. If the model's text fails, a safe template is used.

10. Demo fault controls

Open **Demo controls** from the module rail. Pick a cell (or click a sector on the map), a problem and a severity, then click **Create problem**. Within about two telemetry windows the detector opens an incident and the agents start on their own.

Problem	What happens in the simulator	Expected fix	Team
Capacity surge	An event crowd arrives; PRB use and drops rise	Lower transmit power so users move to a neighbour	RAN Optimisation
Overshoot	A bad config push set the tilt too low; far-user share rises	Restore the tilt	RAN Optimisation
External interference	Uplink RSSI rises, SINR falls	No config fix exists; find the source	RF Engineering
Hardware fault (VSWR)	VSWR and transmit-power alarms, availability falls	Site visit; customers are told a repair is underway	Field Operations

Problem	What happens in the simulator	Expected fix	Team
KPI data dropout	PM files go missing	Re-poll the collector automatically; severity 1 fixes itself	NOC Data Platform

After the incident opens, approve, reject or ask for more information. For field tickets, click **Mark fixed** after dispatch.

Optional switches show the error paths:

- **Red-team the first plan:** forces an aggressive first draft so the safety rules can be seen blocking it.
- **Twin outage:** shows the "not simulated" path.
- **Meta API: primary model down / all models down:** shows the model fallback and the escalation path.

The timed 3-minute video script is in `DEMO.md`.

11. Run, configure and test

Quick start on Windows: double-click `start.bat` in `D:\Meta Ai Heckatho\ran-sentinel`. On the first run it creates the virtual environment, installs the requirements and copies `.env.example` to `.env`. Then it starts the server and opens the console in the browser.

Step by step (Python 3.10 or newer):

```
cd "D:\Meta Ai Heckatho\ran-sentinel"
py -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
notepad .env
py scripts\check_meta.py
py run.py
```

Then open the operations console at http://127.0.0.1:8000/ops/ and the customer app at http://127.0.0.1:8000/connect/. The demo logins are in `.env.example`.

Configuration in `.env` (never committed; listed in `.gitignore`):

Setting	Meaning
`MODEL_API_KEY`	Your Meta Model API key. Leave empty for the offline rules engine
`REVIEWER_MODEL`, `ASSISTANT_MODEL`	Optional model overrides; the file also has PRIMARY_MODEL, FALLBACK_MODEL, LAST_RESORT_MODEL, WONDER_MODEL, SAM_MODEL, VISION_MODEL and VOICE_MODEL, plus token and timeout limits
`MAVERIC_URL`	Optional Maveric service address for the twin adapter
`FCC_USERS_PER_SITE`	Average users per macro site for the FCC estimate (default 1,500)
`OPS_EMAIL`, `OPS_PASSWORD`, `CUSTOMER_EMAIL`, `CUSTOMER_PASSWORD`	Demo logins

Tests: `python -m pytest -q` runs 63 tests in about three minutes. They cover:

- the nine safety rules and M-Plane rollback;
- detection, with no false alerts on a normal day and every fault caught;
- all five fault scenarios end to end;
- the Muse Spark path against a scripted stand-in: tool calls, the repair loop, the evidence loop, the customer-text filter, model fallback and full outage;
- the FCC check, the ZIP-aware assistant, the reviewer and reporter agents and the TelecomTS pipeline;
- the SAM and voice request formats against a mock HTTP server, photo evidence and voice-intent hints;
- the Muse Wonder brief and handover, and the three-tier fallback;
- the alarm monitor fields and acknowledge.

12. Verified and not yet verified

Item	Status	Evidence
Muse Spark 1.3 and 1.2: chat, tool calls, JSON	Verified on a real key	`check_meta.py` on your PC, Oct 28

Item	Status	Evidence
Muse Spark 1.1, Muse Wonder 1.0, Spark vision	Verified on a real key	`check_meta.py`, Oct 28; vision described the test image correctly
SAM 3.1	Verified on a real key	Returned one box with confidence for the generated test image
Muse Voice Transcribe 1.0	Request accepted; accuracy not measured	Tested with a generated tone only, which gives an empty transcript. Next: `check_meta.py --audio note.wav` with a spoken note
SAM on a real site photo	Not yet	Next: `check_meta.py --image site.jpg`
Detection method	Verified offline	TelecomTS benchmark on your PC; simulator benchmark 48 of 48, zero false alarms
End-to-end scenarios, safety, rollback	Verified in tests	63 automated tests pass
Console layout	Verified	Screenshots at 1440, 1024 and 390 px with no JavaScript errors
A full live demo run with the key	Not yet	Do one practice run before recording

Honest limits

- **Simulated network.** 36 cells in Seattle with a small, explainable KPI model. Northstar Connect is fictional.
- **TelecomTS against live data.** TelecomTS checks the detection method on real 5G testbed telemetry (per-UE KPIs every 100 ms). The live demo runs on simulated cell-level KPIs every 15 minutes, because no open dataset offers a cell cluster with config control.
- **Twin.** The what-if runs on a copy of the simulator. The Maveric adapter only checks that the service is running.
- **Open M-Plane.** Simplified O-RAN-style XML goes to an in-process simulated radio unit, not to real hardware or the open-mplane C stack.

- **Support levels.** Hypothesis support levels describe evidence strength, not calibrated probabilities.
- **FCC.** Decision support only, not legal advice.
- **No endorsement.** No Meta, operator or vendor endorsement is implied.

13. Timeline

Date	What was done
Oct 26	Hackathon build window opens.
Oct 27	Reviewed the 22-section design doc and the planning chat. Confirmed the Meta key worked with muse-spark-1.3. Wrote the demo storyboard and the layered workflow v2 (6 layers, 4 bounded loops, error paths). Researched Meta's telecom work and planned the Maveric and Open M-Plane integration. Built the first working app from your UI mockup: simulator, detection, orchestrator, agents, safety, M-Plane, ledger, ops console and customer app.
Oct 28, early	Moved the network to the US: Seattle sites and ZIP codes, bands B66, B2 and B71, Pacific time, the FCC Part 4 check and a Regulatory Compliance desk. Added the Plan Reviewer, the Incident Reporter, the Model card page and the benchmark scripts. Wrote the business case.
Oct 28, morning	You ran the TelecomTS benchmark on your PC. The plain Isolation Forest was weak, so the per-cell Mahalanobis baseline was added; simulator detection reached 48 of 48 with zero false alarms.
Oct 28, 08:08 to 08:35 UTC	Added every useful Meta model: Spark 1.1 as the last-resort tier, Muse Wonder for briefs and the handover, SAM 3.1 and Spark vision for site photos, Muse Voice Transcribe for dictated notes and spoken questions. You ran `check_meta.py` and every model passed.

Date	What was done
Oct 28, 10:23 to 11:45 UTC	Checked the work against Meta's public hackathon requirements. Redesigned the console in the style of professional OSS tools, with the alarm monitor, cell explorer, tabbed incident workspace, agent console, approvals and status bar. Added LICENSE, SUBMISSION.md, DEMO.md, start.bat and the alarm test (63 tests). Wrote the requirements check.
By Nov 3	Your target: practice run, public GitHub repo, 3-minute video.
Nov 4, 9:00 PM ET	Submission deadline (Nov 5, 7:30 AM in Sri Lanka). Judging runs Nov 4 to 8; finals are on Nov 9.

Related documents: the hackathon requirements check and the business case. Source files: `README.md`, `SUBMISSION.md`, `DEMO.md` and `docs/workflow.mmd`.